

Choose one domain

Supply Chain Risk, Social Network, Job Recommendation, Learning/Skill Graph, Fraud Detection, Movie Recommendation, or another approved graph-oriented problem.

Your project must contain at least

5+ node types

5+ relationship types

Show more

Yes. For learning, I recommend **Job Recommendation / Career Skill Graph** because it naturally demonstrates nodes, relationships, multi-hop traversal, recommendations, and Python → Neo4j integration.

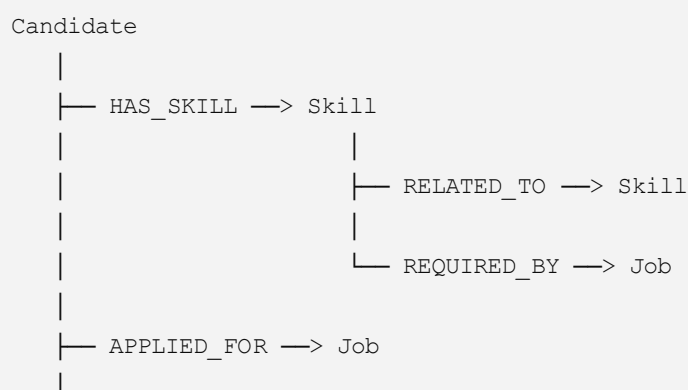
We can build it as a small portfolio project:

CareerGraph — Job Recommendation using Skills, Jobs, Candidates and Companies

You will learn Neo4j step-by-step rather than just copying a finished project.

1. What we will build

A graph like:



```
└─ WORKED_AT ─> Company
```

```
Job ─> POSTED_BY ─> Company
```

```
Job ─> REQUIRES ─> Skill
```

```
Job ─> LOCATED_IN ─> Location
```

```
Company ─> LOCATED_IN ─> Location
```

This gives us **at least 6 node types**:

1. Candidate
2. Skill
3. Job
4. Company
5. Location
6. Industry

And **8 relationship types**:

1. HAS_SKILL
2. REQUIRES
3. RELATED_TO
4. APPLIED_FOR
5. WORKED_AT
6. POSTED_BY
7. LOCATED_IN
8. BELONGS_TO

So your assignment requirements are comfortably covered.

2. Project structure

Create this folder:

```
career-graph/  
├──  
│   ├── data/  
│   │   ├── candidates.csv  
│   │   ├── skills.csv  
│   │   ├── jobs.csv  
│   │   ├── companies.csv  
│   │   ├── locations.csv  
│   │   └── industries.csv  
│   └──  
└── src/
```

```
|   ├── __init__.py
|   ├── db.py
|   ├── create_nodes.py
|   ├── create_relationships.py
|   ├── crud.py
|   └── queries.py
|
| ├── .env
| ├── .gitignore
| ├── requirements.txt
| └── main.py
```

Don't create all of these immediately. We'll build them one at a time.

3. Step 1 — Install Neo4j

You have two choices:

Option A — Neo4j Aura

This is easier because you don't need to install Neo4j locally.

Create a free Neo4j Aura database and obtain:

```
URI
USERNAME
PASSWORD
```

For example:

```
NEO4J_URI=neo4j+s://xxxxxxxxx.databases.neo4j.io
NEO4J_USERNAME=neo4j
NEO4J_PASSWORD=your_password
```

Option B — Neo4j Desktop

You can also install Neo4j Desktop and create a local database.

For local development you might have:

```
NEO4J_URI=bolt://localhost:7687
NEO4J_USERNAME=neo4j
NEO4J_PASSWORD=your_password
```

For learning, either is fine.

4. Step 2 — Create Python environment

From your project directory:

Bash

```
python -m venv venv
```

Windows:

Bash

```
venv\Scripts\activate
```

Then:

Bash

```
pip install neo4j python-dotenv pandas
```

Create `requirements.txt`:

```
neo4j
python-dotenv
pandas
```

Later we can add FastAPI if you want to expose this as an API.

5. Step 3 — `.env`

Create:

```
.env
```

Put:

```
NEO4J_URI=bolt://localhost:7687
NEO4J_USERNAME=neo4j
NEO4J_PASSWORD=your_password
```

If you're using Aura:

```
NEO4J_URI=neo4j+s://xxxxxxx.databases.neo4j.io
NEO4J_USERNAME=neo4j
```

Do not commit `.env` to Git.

Create `.gitignore`:

```
venv/  
.env  
__pycache__/  
*.pyc
```

6. Step 4 — First Python → Neo4j connection

Create:

```
src/db.py
```

Start with this:

Python

Run

```
import os  
  
from dotenv import load_dotenv  
from neo4j import GraphDatabase  
  
load_dotenv()  
  
class Neo4jConnection:  
  
    def __init__(self):  
        self.uri = os.getenv("NEO4J_URI")  
        self.username = os.getenv("NEO4J_USERNAME")  
        self.password = os.getenv("NEO4J_PASSWORD")  
  
        self.driver = GraphDatabase.driver(  
            self.uri,  
            auth=(self.username, self.password)  
        )  
  
    def close(self):  
        self.driver.close()
```

Then create:

```
main.py
```

Python

Run

```
from src.db import Neo4jConnection

db = Neo4jConnection()

print("Connected to Neo4j")

db.close()
```

Run:

Bash

```
python main.py
```

If you get:

```
Connected to Neo4j
```

your first milestone is complete.

7. Step 5 — Understand a node

Before creating 50+ nodes, understand one node.

In Neo4j Browser:

cypher

```
CREATE (:Skill {
  name: 'C#'
});
```

Then:

cypher

```
MATCH (s:Skill)
RETURN s;
```

You should see:

```
Skill
  name: C#
```

Now create another:

cypher

```
CREATE (:Skill {
  name: 'Angular'
});
```

and:

cypher

```
CREATE (:Skill {
  name: 'Azure'
});
```

8. Step 6 — Understand relationships

Create:

cypher

```
MATCH (c:Skill {name: 'C#'})
MATCH (a:Skill {name: 'Angular'})
CREATE (c)-[:RELATED_TO]->(a);
```

Now:

cypher

```
MATCH (s1:Skill)-[r:RELATED_TO]->(s2:Skill)
RETURN s1, r, s2;
```

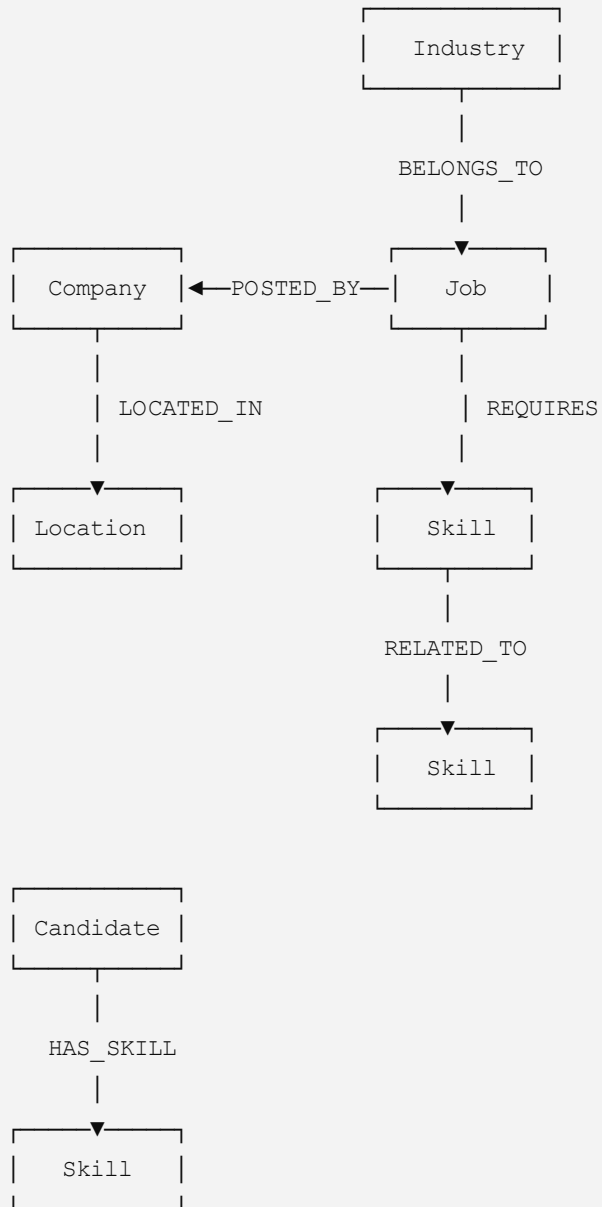
Conceptually:

C# —RELATED_TO—> Angular

This is the most important concept to understand before moving to Python.

9. Step 7 — Define our complete graph model

Our graph will eventually look roughly like this:



10. Step 8 — Create realistic data

We'll eventually create something like:

Candidates

```
C001 Krishna
C002 Rahul
C003 Priya
...
C015 ...
```

Skills

S001 C#
S002 .NET
S003 Angular
S004 Python
S005 Azure
S006 SQL
S007 Docker
S008 Kubernetes
S009 Machine Learning
S010 Generative AI
...

Jobs

J001 Senior .NET Developer
J002 Azure Developer
J003 Full Stack Developer
J004 AI Engineer
J005 Data Engineer
...

Companies

COMP001 Microsoft
COMP002 Accenture
COMP003 Capgemini
COMP004 Deloitte
...

Locations

LOC001 Hyderabad
LOC002 Bangalore
LOC003 Chennai
LOC004 Pune
LOC005 Delhi

Industries

IND001 Banking
IND002 Insurance
IND003 Healthcare
IND004 Retail
IND005 Technology

We can generate enough realistic data to satisfy:

50+ nodes
75+ relationships

For example:

Node type	Count
Candidate	10
Skill	15
Job	15
Company	5
Location	5
Industry	5
Total	55

Then relationships can easily exceed 75.

11. Step 9 — Use Python to create nodes

Instead of manually executing 55 `CREATE` statements, Python will insert them.

Example:

Python

Run

```
from src.db import Neo4jConnection

db = Neo4jConnection()

def create_skill(tx, name):

    query = """
    CREATE (s:Skill {
        name: $name
    })
    """

    tx.run(query, name=name)
```

```
skills = [
    "C#",
    ".NET",
    "Angular",
    "Python",
    "Azure",
    "SQL",
    "Docker",
    "Kubernetes"
]

with db.driver.session() as session:

    for skill in skills:

        session.execute_write(
            create_skill,
            skill
        )

db.close()
```

Notice this:

cypher

```
{name: $name}
```

That's a **parameterized query**.

We are not doing:

Python

Run

```
query = f"""
CREATE (:Skill {{name: '{skill}'}})
"""
```

Parameterized queries are an important Neo4j concept.

12. Step 10 — Create relationships from Python

Example:

Python

Run

```
query = """
MATCH (c:Candidate {id: $candidate_id})
MATCH (s:Skill {name: $skill})
MERGE (c)-[:HAS_SKILL]->(s)
"""

session.run(
    query,
    candidate_id="C001",
    skill=".NET"
)
```

This produces:

```
Candidate C001
  |
  | HAS_SKILL
  ▼
.NET
```

13. Step 11 — Learn CRUD

Your project specifically requires CRUD.

We should implement each separately.

CREATE

cypher

```
CREATE (s:Skill {
    id: $id,
    name: $name
})
```

READ

cypher

```
MATCH (s:Skill)
RETURN s
```

UPDATE

cypher

```
MATCH (s:Skill {id: $id})
SET s.name = $name
RETURN s
```

DELETE

cypher

```
MATCH (s:Skill {id: $id})
DELETE s
```

But there's an important Neo4j concept here:

If a node has relationships, this:

cypher

```
DELETE s
```

can fail.

You may need:

cypher

```
DETACH DELETE s
```

Understanding **why** is part of the learning.

14. Step 12 — Your 10+ meaningful Cypher queries

This is where the project becomes interesting.

We shouldn't make 10 meaningless queries just to satisfy the assignment.

We'll create useful queries.

Query 1 — Find all skills

cypher

```
MATCH (s:Skill)
RETURN s
```

Query 2 — Find candidate skills

cypher

```
MATCH (c:Candidate {id: $candidate_id})
      -[:HAS_SKILL]->(s:Skill)

RETURN c.name, collect(s.name) AS skills
```

Query 3 — Jobs requiring a particular skill

cypher

```
MATCH (j:Job)-[:REQUIRES]->(s:Skill)
WHERE s.name = $skill

RETURN j.title
```

Example parameter:

```
skill = "Azure"
```

Query 4 — Jobs at a company

cypher

```
MATCH (j:Job)-[:POSTED_BY]->(c:Company)
WHERE c.name = $company

RETURN j.title
```

Query 5 — Candidates who know .NET

cypher

```
MATCH (c:Candidate)-[:HAS_SKILL]->(s:Skill)
WHERE s.name = ".NET"

RETURN c.name
```

Query 6 — Jobs in Hyderabad

cypher

```
MATCH (j:Job)-[:LOCATED_IN]->(l:Location)
WHERE l.name = $location

RETURN j.title
```

Query 7 — Candidate → Skills → Jobs

Now we start doing graph traversal:

cypher

```
MATCH (c:Candidate {id: $candidate_id})
      -[:HAS_SKILL]->(s:Skill)
      <-[:REQUIRES]- (j:Job)

RETURN c.name, s.name, j.title
```

This is a **2-hop traversal**.

```
Candidate
  ↓
Skill
  ↓
Job
```

15. Step 13 — Multi-hop traversal

This is one of the most important requirements.

Suppose:

```
Candidate
  ↓
HAS_SKILL
  ↓
Skill
  ↓
REQUIRED_BY
  ↓
```

```
Job
  ↓
POSTED_BY
  ↓
Company
  ↓
LOCATED_IN
  ↓
Location
```

That's multiple hops.

Cypher:

cypher

```
MATCH path =
  (c:Candidate {id: $candidate_id})
  -[:HAS_SKILL]->
  (s:Skill)
  <-[:REQUIRES]-
  (j:Job)
  -[:POSTED_BY]->
  (company:Company)
  -[:LOCATED_IN]->
  (location:Location)

RETURN path
```

This is where Neo4j becomes much more interesting than a traditional relational database query.

16. Step 14 — Build a job recommendation query

This will be our main project feature.

Suppose:

Candidate has:

```
.NET
C#
Angular
Azure
SQL
```

And a job requires:

.NET
C#
Azure
Docker
Kubernetes

We can find matching skills.

For example:

cypher

```
MATCH (c:Candidate {id: $candidate_id})
      -[:HAS_SKILL]->(s:Skill)
      <-[:REQUIRES]-(j:Job)

WITH c, j, count(s) AS matchingSkills

RETURN
  c.name AS candidate,
  j.title AS job,
  matchingSkills
ORDER BY matchingSkills DESC
```

Now we have a simple graph-based recommendation mechanism.

17. Step 15 — Improve recommendation

Eventually we'll calculate:

```
Candidate skills
      ↓
Matching job requirements
      ↓
Number of matching skills
      ↓
Location
      ↓
Industry
      ↓
Recommendation
```

For example:

Candidate: Krishna

Job

Matching Skills

```
-----
Azure Developer          4
Senior .NET Developer    5
Full Stack Developer     4
Data Engineer           2
AI Engineer              1
```

We won't simply hard-code this ranking. We'll build the Cypher logic that calculates the matching relationships.

18. Step 16 — More advanced query

We can calculate both matching and missing skills.

cypher

```
MATCH (c:Candidate {id: $candidate_id})
MATCH (j:Job)

OPTIONAL MATCH (j)-[:REQUIRES]->(required:Skill)
OPTIONAL MATCH (c)-[:HAS_SKILL]->(candidateSkill:Skill)

WITH
    c,
    j,
    collect(DISTINCT required.name) AS requiredSkills,
    collect(DISTINCT candidateSkill.name) AS candidateSkills

RETURN
    j.title,
    [skill IN requiredSkills
     WHERE skill IN candidateSkills] AS matchingSkills,
    [skill IN requiredSkills
     WHERE NOT skill IN candidateSkills] AS missingSkills
```

This gives you something very useful:

Job:
Senior Azure Developer

Matching:
.NET
C#
Azure
SQL

Missing:

That's a much better portfolio demonstration.

19. Step 17 — Python CRUD layer

Eventually your `crud.py` can have:

Python

Run

```
def create_candidate(...):  
    pass  
  
def get_candidate(...):  
    pass  
  
def update_candidate(...):  
    pass  
  
def delete_candidate(...):  
    pass  
  
def add_skill_to_candidate(...):  
    pass  
  
def remove_skill_from_candidate(...):  
    pass  
  
def recommend_jobs(...):  
    pass
```

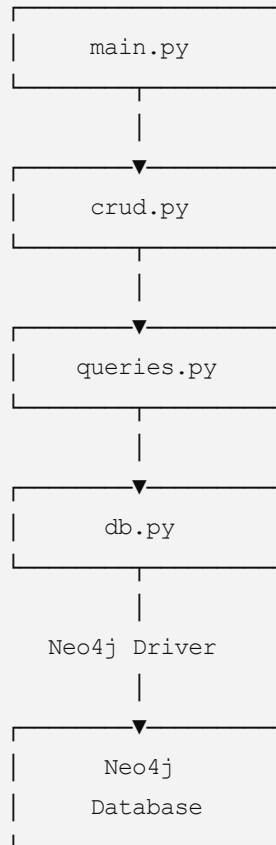
Then:

```
main.py  
|  
▼  
crud.py  
|  
▼  
db.py  
|  
▼  
Neo4j
```

This gives you a clean architecture rather than one giant Python file.

20. Step 18 — Final project architecture

Eventually:



21. Requirements checklist

By the end, we'll explicitly verify your assignment:

Requirement	Our implementation
Domain	Job Recommendation
5+ node types	✓ 6
5+ relationship types	✓ 8
50+ nodes	✓ ~55+
75+ relationships	✓ 100+

Requirement	Our implementation
Python → Neo4j	✓ Neo4j Python Driver
CREATE	✓
READ	✓
UPDATE	✓
DELETE	✓
10+ Cypher queries	✓
Multi-hop traversal	✓
Parameterized queries	✓
<code>.env</code>	✓
<code>requirements.txt</code>	✓

Recommended learning sequence

Don't build the entire project in one shot.

Follow this sequence:

Day/Step 1

- |
- ├ Neo4j basics
- ├ Nodes
- ├ Relationships
- └ MATCH / CREATE

↓

Step 2

- |
- ├ Labels
- ├ Properties
- ├ MERGE
- └ DELETE / DETACH DELETE

↓

Step 3

- |
- ├ Python Neo4j Driver
- ├ .env
- └ Connection

↓

Step 4

- |
- ├ Create 50+ nodes
- └ Create 75+ relationships

↓

Step 5

- |
- ├ CRUD
- └ Parameterized queries

↓

Step 6

- |
- ├ 10+ meaningful Cypher queries
- └ Multi-hop traversal

↓

Step 7

- |
- └ Job recommendation algorithm

↓

Step 8

- |
- ├ Clean Python architecture
- ├ README
- └ Project demonstration

I suggest we start with Step 1 only: I'll teach you the Neo4j fundamentals using **5 Candidates + 8 Skills + 5 Jobs**, and you'll manually execute the Cypher queries in Neo4j Browser before we introduce Python. That will make the later Python integration much easier to understand.

Is this conversation helpful so far?

ChatGPT can make mistakes. Check important info.